RaPToR-Lite: A Capability-Grounded Natural-Language Task Creation and Verification Layer for ROS 2 Robots — A House-Sitter Case Study

[STUDENT ID REQUIRED]

SEIoT MSc Final Project Report 2026

University College London

ABSTRACT

Domestic mobile robots often sit idle despite possessing resources that could support other household services. This dissertation presents RaPToR-Lite, a lightweight capability-grounded layer for creating and checking natural-language robot tasks, evaluated through a House-Sitter case study. The system separates constrained language interpretation from a typed TaskSpec, runtime capability grounding, explicit verification, resource-aware admission and deterministic execution. A common RobotBackend supports a seeded House2D experimental environment and a ROS 2 deployment backend targeting the iRobot Create 3; the latter was interface- and mock-tested, but not validated on physical hardware.

Three formal experiments were conducted with independent static oracles. On 240 paired decision cases, the full system made 240 correct accept, reject or clarify decisions (100%, 95% CI 98.4–100.0%), improving by 20.0 percentage points over removal of capability grounding and by 33.3 points over removal of verification. Across 320 controlled utterances, decision accuracy was 98.4%, but exact TaskSpec match was 85.0% and end-to-end correctness was 79.1%; synonym forms achieved only 55.0%, identifying canonicalisation as a principal weakness. On 300 held-out stochastic House2D seeds, 61.7% of missions completed, 22.0% were safely deferred and 16.3% terminated safely at a blocked route. No genuine unexpected execution failure occurred, although primary anomaly-detection F1 was only 0.403. Route optimisation reduced route cost by 4.74 units on average without changing completion. All 40 scientific replays matched.

The findings show that explicit capability and verification boundaries can substantially improve task-level safety decisions in a controlled setting, while language coverage and onboard-equivalent monitoring remain limiting factors. RaPToR-Lite is therefore evidence for a bounded mediation architecture, not for open-world language understanding or physical-robot reliability.

AUTHOR KEYWORDS

Natural-language robot programming; capability grounding; task verification; domestic service robots; Digital Twin; ROS 2; reproducible robotics.

TABLE OF CONTENTS

1. INTRODUCTION

Domestic robots are no longer unusual household objects, but their useful behaviour is often much narrower than their hardware suggests. A robotic vacuum may contain a mobile base, odometry, inertial sensing, hazard sensors, wireless communication and a battery-management system, yet spend most of its time waiting to clean. Longitudinal studies of robotic vacuums show that their role is shaped by everyday routines and appropriation rather than by technical functionality alone [21]. More recently, Shiokawa et al. elicited over one hundred uses for domestic robots during otherwise idle time, including monitoring and assistance activities [53]. This creates an attractive engineering question: can a user safely repurpose an available mobile robot through a task-level interface, without treating unrestricted language as executable control?

Natural language is convenient precisely because it leaves detail implicit. A person may request, “check the kitchen and living room, then come back”, while assuming a shared meaning for check, a known room map, suitable sensors, enough battery, a safe route and an obvious recovery policy. A robot cannot safely inherit those assumptions. Work on grounded language has mapped phrases to spatial entities, actions and planning representations [29,34,42,55], while recent language-model systems can propose plans, programs or visuomotor actions [16,30,31,37]. These approaches demonstrate the value of linguistic task creation, but they also make the execution boundary important. A fluent plan may still name an absent capability, omit a timeout, contradict the robot state, or be unsafe to begin with.

RaPToR-Lite addresses that boundary with a deliberately small architecture. Natural language is first converted into a typed, inspectable TaskSpec. The plan is grounded against a capability profile describing what the selected robot backend actually exposes. A deterministic Verifier reports errors and warnings, explains the decision and can propose bounded repairs for selected structural defects. A separate resource policy assesses whether the task can start while preserving a return reserve. Only an approved task, after explicit confirmation, reaches an executor that checks the plan again. The design follows the broader principle that flexible task generation should be separated from constrained robot control [9,15], but applies it to a practical household workflow rather than claiming formal proof of arbitrary behaviour.

The House-Sitter application provides a concrete case. A task can patrol named rooms, collect temperature, humidity, obstacle, accessibility and object observations, identify anomalies, update a room-oriented Digital Twin, retain history and produce actionable feedback. Route optimisation chooses a lower-cost ordering over the fixed House2D topology. Resource admission can defer a mission whose estimated cost and safe-return reserve exceed the current battery. These behaviours share a RobotBackend contract. House2D is the experimental backend used for controlled evaluation. Create3ROS2Backend is a deployment backend that discovers the live ROS graph and maps only observed Create 3-compatible topics, actions and services; semantic room navigation is unavailable unless a valid navigation provider supplies it.

This separation matters for scientific as well as operational reasons. A simulator exposes complete world state, but a detector on a real robot would not know scenario seeds, event identifiers or truth labels. The final experimental pipeline therefore distinguishes WorldGroundTruth and run provenance from RobotObservation. Detectors consume only onboard-equivalent observations. Randomised variables are audited for actual consumption, and an unused obstacle-position variable was removed before any RQ3 data were admitted. Formal cases and labels were materialised before execution using independent deterministic oracles that do not import the planner, Verifier or detector. Pilot data were excluded. Raw records contain hashes, seeds, conditions and paired identifiers; scientific replay excludes volatile timestamps and paths while retaining every field relevant to conclusions.

The dissertation asks three research questions:

- RQ1: To what extent do capability grounding and explicit verification improve the correctness and safety of task-level decisions in natural-language robot task creation?
- RQ2: How reliably can a constrained natural-language interface translate diverse household task expressions into correct and executable structured robot tasks?
- RQ3: How does the integrated House-Sitter system perform under controlled stochastic household conditions in terms of task completion, safety, monitoring accuracy, route efficiency, Digital Twin correctness and reproducibility?

Four contributions follow. First, RaPToR-Lite is a lightweight capability-grounded task-creation architecture that keeps language interpretation outside deterministic execution. Second, it provides a robot-aware mediation layer combining runtime capability profiles, explainable verification, confirmation, bounded repair and resource-aware admission. Third, it integrates patrol, environmental monitoring, route optimisation, Digital Twin history and robot feedback behind experimental and ROS 2 deployment backends. Fourth, it provides a reproducible evaluation method using independent oracles, pre-specified corpora, held-out seeds, paired ablations, provenance manifests and deterministic scientific replay.

The claims are intentionally bounded. The natural-language planner is constrained and deterministic, not a general conversational agent. House2D is a simplified research environment, not a photorealistic simulator or a model of sensor physics. The Create 3 backend is implemented and interface-tested, but physical robot operation, real-house navigation and physical safety were not demonstrated.

Accordingly, the thesis evaluates whether explicit mediation improves decisions within a controlled benchmark and whether the resulting House-Sitter behaves predictably under controlled stochastic conditions. It does not claim open-world language reliability or real-world deployment readiness.

The remainder of the report reviews domestic robotics, grounded language, verification, smart-home monitoring, Digital Twins, ROS 2 and reproducibility. It then describes the architecture and both backend boundaries, presents the three formal evaluations, and discusses why strong decision safety coexists with weaker semantic canonicalisation and monitoring accuracy.

2. LITERATURE REVIEW

2.1 Domestic Robots Beyond Their Primary Function

Domestic service robots occupy a distinctive human–robot interaction setting. They operate near non-experts, become embedded in routines and encounter spaces that were not engineered for robotics. Goodrich and Schultz characterise human–robot interaction through autonomy, information exchange, team organisation and task context [25]. Social-robot surveys similarly show that users infer roles and intentions from a robot's behaviour, even when its mechanical capability is limited [20]. In the home, acceptance therefore depends not only on whether a robot can perform a motion, but on whether its behaviour is intelligible, controllable and useful.

Studies of robotic vacuums make this point concrete. Forlizzi and DiSalvo found that Roomba use altered cleaning practices and household relationships over time [21]. Dautenhahn et al. reported that users distinguished a robot as friend, assistant or butler, implying different expectations of initiative and responsibility [13]. Work on assistive agents also links acceptance to perceived usefulness, ease of use, trust and social influence [28]. These findings caution against presenting task creation as a purely technical translation problem: an accepted command must still expose what the robot will do and why it may refuse.

The “Beyond Vacuuming” study provides the immediate project motivation. Shiokawa et al. combined a user survey with HCI/HRI expert input to identify activities that domestic robots might perform during idle time [53]. Many suggestions rely on properties already present in mobile robots: movement between locations, observation, temporal repetition and communication. A house-sitting patrol is therefore plausible without adding a manipulator or specialised smart-home installation. It is also a useful stress case because a seemingly simple request spans language, route choice, sensing, state persistence, anomaly interpretation and battery constraints.

This project treats reuse as bounded repurposing, not unrestricted autonomy. Socially assistive robotics shows that benefit need not require physical manipulation [19]. A mobile observer can report conditions, identify a blocked transition and preserve a history without opening doors or moving objects. That narrower role fits a Create 3-class base better than general domestic assistance, and it makes unsupported requests explicit rather than concealing them behind a speculative action generator.

2.2 Natural-Language Interaction with Robots

Robot language understanding differs from conventional text classification because words must ultimately be connected to entities, actions and consequences. Kollar et al. grounded route descriptions in spatial features and paths [34]. Tellex et al. introduced probabilistic grounding graphs that associate linguistic constituents with robot actions and environment objects [55]. Matuszek et al. learned mappings from natural-language commands to a robot control system [42], while Howard et al. connected language to a planner for mobile manipulation [29]. Collectively, this work establishes that useful commands require a representation between surface text and low-level control.

Learning from demonstration offers another route for transferring task knowledge, but correspondence, generalisation and representation remain central design problems [4]. Language can complement demonstration by naming goals and constraints; it does not remove the need to identify which robot actions those words denote.

Grounding also depends on context. Tell Me Dave interpreted manipulation instructions with respect to a scene and task [44]. Vision-and-Language Navigation and ALFRED extended the challenge to long-horizon navigation and household activities in visually grounded environments [3,54]. These benchmarks reveal a gap between recognising an intent and producing a fully correct action sequence. The same distinction motivates separate decision, intent, TaskSpec-exact and end-to-end metrics in RQ2.

Language variation is not merely noise. Synonyms may imply the same activity but trigger different lexical rules; unordered room lists may preserve a set-valued goal while changing route choice; an explicit order may be semantically binding. Bisk et al. argue that linguistic meaning is grounded through experience and interaction rather than text in isolation [6]. Tellex et al. likewise frame language-using robots around links among symbols, perception, action and dialogue [56]. A constrained interface can make these links auditable, but its vocabulary remains a genuine coverage limit.

Ambiguity requires a decision, not a guessed completion. Human–robot dialogue can ask questions that reduce uncertainty in grounded interpretation [57]. Marge et al. identify clarification, error recovery and common evaluation practice as central research needs for spoken robot interaction [40]. RaPToR-Lite therefore gives clarify equal status with accept and reject. Clarification indicates that a supported task may exist but the current wording does not determine it safely; rejection indicates that the request is invalid, unsafe or unsupported under the available profile.

2.3 Structured Tasks and Capability Grounding

Automated planning represents goals, state and admissible actions explicitly [23]. Behaviour Trees offer another modular representation in which control flow and recovery remain inspectable [11]. RaPToR-Lite does not implement a general PDDL planner or full Behaviour Tree runtime. It adopts the narrower lesson: language output should become typed data with explicit steps, parameters, timeout and failure semantics before it is executable.

Capability grounding asks whether those structured steps correspond to operations available on the selected robot. In prior grounded-language systems, possible actions are often fixed by a benchmark or learned affordance model. SayCan combines language-model proposals with value functions expressing what a robot can do in its current setting [31]. Its results demonstrate that action feasibility can correct plausible but impractical language plans. RaPToR-Lite makes the capability side explicit as a versioned profile. This is less expressive than a learned affordance model, but it is inspectable, deterministic and able to represent unavailable without inventing support.

Capability and validity are related but different. A robot may expose navigation while a plan still omits a timeout or safe-return action. Conversely, a well-formed plan may request a sensor that is not present on the connected backend. Keeping the Capability Registry and Verifier separate permits paired counterfactual evaluation of these roles in RQ1. It also supports deployment discovery: a Create 3 interface is advertised only when the required ROS graph entity and type are observed.

2.4 Language Models and Embodied Planning

Large language models have widened the set of possible robot interfaces. Zero-shot planning extracts action sequences from model knowledge [30]; Code as Policies produces programs that call robot APIs [37]; Toolformer studies learned use of external tools [51]. Multimodal systems such as PaLM-E and RT-2 connect language, images and actions [16,60], while RT-1 demonstrates large-scale real-robot policy learning [8]. These systems provide important evidence that language can support compositional control.

They do not remove the interface problem. Generated code is constrained by the APIs it can call, and a model's semantic plausibility does not establish a robot's actual capability or the safety of a particular execution. Many systems also require data and compute far beyond an MSc implementation. RaPToR-Lite deliberately explores a complementary point in the design space: a deterministic local parser for a restricted household language, paired with explicit verification and a backend boundary. This sacrifices open-ended coverage to isolate the value of mediation and to permit independent, fully materialised oracles.

2.5 Verification, Runtime Assurance and Explainability

Safety mechanisms for autonomous systems range from formal synthesis to runtime enforcement. Kress-Gazit et al. review robot controller synthesis in which guarantees depend on explicit specifications and assumptions [35]. Luckcuck et al. show that formal methods address requirements, software, environment and system models, but also identify difficulties scaling evidence across autonomous robotic systems [38]. Seshia et al. similarly argue that verified artificial intelligence requires specifications and evidence covering the environment and learning-enabled components [52].

Runtime assurance provides a pragmatic architectural pattern. SOTER separates an advanced controller from simpler safety monitors and fallback behaviour [15]. Shielding constrains actions that violate a safety specification [2]. RaPToR-Lite is not a formal safety certificate and its Verifier is not a complete runtime shield. It applies the separation at task level: a flexible or fallible creator cannot directly command motion; deterministic checks cover the declared schema, capability, sequencing and resource invariants before execution.

Explainability here is operational rather than post-hoc. The Verifier returns issue codes, paths, severity, messages and suggested actions. A user can see whether a plan was rejected because a skill is unavailable, a parameter is invalid, a timeout is absent or a safe-return step is missing. This is narrower than explaining a learned model's internal reasoning, but it is directly connected to the gate that prevents execution. The same explicit issues form an auditable basis for the RQ1 error taxonomy.

2.6 Smart Homes and Environmental Monitoring

Smart-home research commonly combines distributed sensors, context modelling and services [1,10,14]. CASAS demonstrates how instrumented environments can support activity recognition and longitudinal datasets [12]. Context-aware computing surveys emphasise a pipeline from acquisition through modelling and reasoning to delivery [49]. Such environments can provide broad coverage, but require installed infrastructure, calibration and governance.

A mobile domestic robot offers a different trade-off. It can move a smaller sensor set between rooms, but observations are sequential rather than simultaneous and may be interrupted by navigation or battery limits. The House-Sitter workflow uses this mobile-observer model: it records what was observed at a particular room and time, distinguishes missing observations from normal values, and carries uncertainty into Digital Twin updates. The simplified House2D sensors are intentionally not treated as equivalent to a deployed smart home.

Smart-home benefits also introduce privacy, reliability and user-control risks [59]. This dissertation does not evaluate privacy or human acceptance empirically, but it avoids embedding truth labels in user feedback and preserves provenance for each update. A deployment would require

additional access control, retention policy and household study.

2.7 Digital Twins for Household State

Digital Twin terminology spans high-fidelity lifecycle models and simpler synchronised digital representations. Early formulations described virtual counterparts linked to complex physical assets [24,26]. Reviews distinguish digital models, shadows and twins by the direction and automation of information exchange [36]. Fuller et al. identify data integration, synchronisation and validation as continuing challenges [22].

RaPToR-Lite uses “Digital Twin” in a modest application sense: a structured, revisioned representation of room observations with provenance and history. It does not claim predictive physical fidelity. Updates occur only from valid RobotObservation records, and missing or untrusted observations do not silently overwrite known state. History and diff functions make confirmed changes inspectable. This limited use still adds value by separating current believed state from raw sensor events and by connecting alerts to the revision that motivated them [18].

2.8 ROS 2 and Reproducible Robotics

ROS 2 provides a modular graph of robot software with distributed discovery, typed communication and Quality of Service (QoS) controls [39]. Performance and delivery depend on middleware and deployment configuration [41]. Topics, services and actions have different interaction semantics [48], while QoS compatibility affects whether state is received and remains fresh [47]. A deployment backend must therefore discover both names and types rather than equating an intended topic name with an available capability.

Reproducibility is particularly difficult in robotics because software, environments and hardware interact. Reviews of AI research find that datasets, seeds, code and experimental detail are often incomplete [27]. Plesser notes that reproducibility and replicability are used inconsistently unless the repeated artefacts and conditions are stated [50]. Recent robotics guidance similarly argues for explicit evidence and procedures [7]. The methodology in this dissertation responds with fixed versions, materialised cases, independent labels, held-out seeds, paired conditions, immutable raw records and a scientific replay projection that separates behaviour from volatile provenance.

2.9 Research Gap

Prior work demonstrates sophisticated language grounding, large-model planning, formal assurance and smart-home sensing. These strands leave a practical gap for a small robot application: an inspectable layer that accepts constrained household language, grounds it in the capabilities actually available at runtime, explains why a task is not executable, applies resource admission, and then delegates to either a reproducible experimental backend or a ROS 2 deployment backend. The contribution is not a new foundation model or a proof of physical safety. It is the integration and controlled evaluation of this mediation boundary, including negative results about language canonicalisation and monitoring accuracy.

3. SYSTEM DESIGN AND IMPLEMENTATION

3.1 Design Goals

Five goals shaped RaPToR-Lite. First, language creation and robot execution must remain separate. A text parser may propose a task but cannot publish velocity commands or invoke robot actions. Second, support must be capability-grounded: an action is available only when the selected backend's profile declares the required skill and observations. Third, failure must be explicit. Missing topics, stale sensors, low battery, invalid parameters and timeouts produce named issues rather than a simulated success. Fourth, evaluation must be deterministic and inspectable. A seeded backend, typed artefacts and replayable records take priority over visual realism. Fifth, the experimental backend must not become a hidden fallback for deployment. Selecting Create3ROS2Backend never substitutes House2D when a graph entity is absent.

These goals imply a pipeline rather than a monolithic agent. The parser handles linguistic variation; TaskSpec records the proposed programme; the Capability Registry records affordances; the Verifier applies invariants; resource admission accounts for the current state; and the executor interacts with a RobotBackend. The structure follows the separation between deliberation and bounded control found in layered robotics [9], but uses ordinary typed Python data so every transition is serialisable and testable.

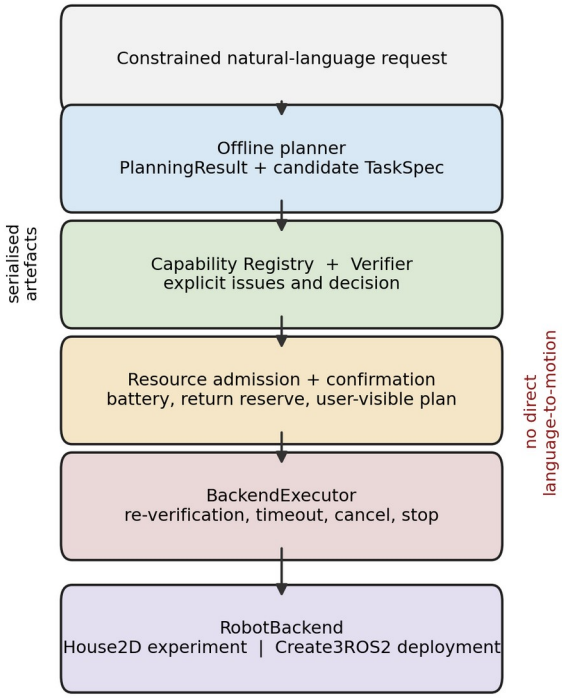

Figure 1. RaPToR-Lite architecture. Natural language produces a candidate TaskSpec, which is grounded and verified before resource admission, confirmation and backend execution. Evidence and feedback are derived from the same explicit artefacts; no language component directly controls motion.

3.2 RaPToR-Lite Architecture

The system is implemented as a small Python package. Pydantic models define strict wire representations. The offline planner emits a PlanningResult containing the original text, status, issues and an optional TaskSpec. CapabilityRegistry loads a backend profile. Verifier consumes the TaskSpec and profile and returns a VerificationReport. A resource-policy function combines the approved task with current battery and estimated movement/inspection cost. BackendExecutor performs a second verification immediately before dispatch and records an ExecutionResult. The House-Sitter layer turns observations into detections, Digital Twin changes, alerts and a monitoring report.

The boundaries are data boundaries. Each stage receives a serialisable input and returns a serialisable output rather than sharing mutable simulator state. This permits the same candidate TaskSpec to be evaluated under full, no-grounding and no-verifier counterfactual conditions in RQ1 without executing rejected work. It also permits RQ2 to compare planner output with static labels independently of the executor. In RQ3, the seed-generated WorldGroundTruth is stored for scoring, while the detector receives a separate RobotObservation object.

The architecture is “Lite” in two senses. It avoids an external language model and its network, cost and nondeterminism, and it does not attempt a universal robot skill ontology. The cost is restricted language and manually defined capability schemas. This is a deliberate research choice: the work tests whether the mediation layers have measurable value under controlled conditions, not whether a particular large model can improvise household plans.

3.3 TaskSpec

TaskSpec is the executable contract between task creation and verification. It contains a schema version, task identifier, human-readable intent, ordered steps and task-level metadata. Each step names a skill, supplies typed parameters, states a timeout and declares what to do on failure. The model rejects unknown fields rather than ignoring them. This prevents a misspelled safety property or unsupported extension from disappearing during deserialisation.

The House-Sitter vocabulary includes navigation to a room, observation, anomaly detection, Digital Twin update, alert/report generation, return and stop. Ordering is meaningful: observation must follow arrival; detection reads observations; Twin updates read detected or valid observed state; the report summarises recorded evidence. A return-to-start and final stop make completion conditions visible. The TaskSpec does not contain velocity profiles or action-server implementation details. Those remain backend concerns.

This representation is intentionally smaller than PDDL and less compositional than a Behaviour Tree [11,23]. It is sufficient to expose the invariants evaluated here: skill existence, parameter shape, time bounds, failure policy, observation dependencies and safe termination. It also makes confirmation meaningful because the user can inspect a finite step list rather than approve opaque generated code.

3.4 Capability Registry

CapabilityRegistry stores a versioned profile of skills and observation channels. Each skill includes its parameter schema and availability; profiles also identify backend name, version, research/deployment role and validation boundary. The House2D profile declares semantic rooms and simulated sensors used by the case study. Create3ROS2Backend builds a profile from discovery results, so missing graph entities appear under unavailable rather than being inferred from a product name.

Grounding is evaluated before execution and before resource estimation. A valid TaskSpec can therefore be rejected when it requests observe_temperature from a backend without a temperature source, or clarified when named-room navigation cannot be resolved. Conversely, capability presence does not certify the rest of the task. RQ1 separates these effects: removing grounding admits unsupported cases, whereas removing verification admits structurally unsafe or invalid cases.

Profiles are evidence rather than marketing descriptions. They retain the ROS name and type that caused a capability to be exposed, and the deployment-readiness report lists discovered and missing entries. This allows a user to distinguish “Create 3 generally documents an action” from “the connected graph currently offers the expected action type”.

3.5 Natural-Language Planner

OfflineHouseSitterPlanner is deterministic and rule-based. It recognises the bounded activities needed by the study, extracts named rooms and ordering cues, normalises supported observation requests and generates the required return, stop and report structure. It returns one of three task-level decisions. accept means a single supported interpretation produced a candidate TaskSpec; clarify means a plausible task lacks enough information or has conflicting interpretations; reject means the request asks for unsafe or unsupported behaviour.

The planner does not call the Verifier to create its RQ2 ground truth. That distinction became important during pre-execution validity review. The 320 utterances and their expected decisions, intents and normalised TaskSpecs were materialised using a separate deterministic rule specification and then stored as static labels. Dependency guards fail if the oracle imports the planner, Verifier or detector. Thus an exact match measures agreement with an independent target rather than the component scoring its own output.

The controlled grammar supports canonical requests, paraphrases, synonyms, explicit route order, unordered room sets, ambiguity, unsupported requests and attempts to bypass safety. It is not a general semantic parser. In particular, lexical precedence can map the phrase “monitoring task” to patrol before a more specific requested inspection is recognised. This behaviour is retained in the formal results rather than patched after observing RQ2 outcomes.

3.6 Verification and Explainability

Verifier applies four groups of checks. Schema and structural checks ensure required fields, supported schema version and valid step order. Capability checks bind each skill and observation requirement to the selected profile. Parameter checks enforce types, bounds and known enumerations. Safety checks require timeouts, permitted failure policies, safe return where needed and final stop. The report contains an overall decision plus issues with stable codes, severity, field path, explanation and recommended action.

Verification is deterministic and independent of execution. Unsupported and unsafe plans are never executed merely to observe their failure. The RQ1 ablations are read-only counterfactual decision evaluations: a no-grounding condition asks what the decision would have been without capability issues, and a no-verifier condition asks what would happen without verification issues. Both still retain the original ground-truth label and record would_accept, would_reject or would_clarify. This design prevents an ablation from becoming an unsafe robot trial.

Selected errors can receive a safe-repair proposal, such as adding an explicit return/stop sequence or replacing an unsupported default with a supported alternative. The proposal is a new TaskSpec and must be verified again; it is not silently substituted. The implementation and tests demonstrate this mechanism, but the formal raw records do not provide an auditable denominator for a repair-success percentage. The dissertation therefore describes repair as a feature and reports its formal empirical rate as not available.

3.7 Confirmation and Resource Policy

An approved task remains a candidate until confirmation. The confirmation view summarises the intent, route, observations, estimated cost, safe-return reserve, warnings and available repair. This prevents the natural-language surface form from being the only description a user sees before execution. The local demo uses the same artefacts as the command-line path rather than a separate permissive controller.

Resource admission is evaluated after verification because battery estimates are meaningful only for a valid route and skill sequence. The policy compares current battery with estimated task cost, return reserve and safety margin. It can approve, reject or safely defer. A defer is not recorded as mission completion, but neither is it a system crash: the system avoided beginning work that could violate the safe-return constraint.

The resource ablation in RQ3 is non-executing. For a task deferred by the full policy, it calculates whether the task would have been attempted without resource admission and whether that hypothetical attempt would violate the battery/safe-return constraint. The resulting measure is named counterfactual unsafe attempt, not unsafe execution. No task protected by a full-policy defer is executed physically or in simulation under the unsafe condition.

3.8 House-Sitter Workflow

The application workflow is patrol, observe, detect, update and report. After a baseline snapshot, the robot visits the requested rooms. At each visit, the backend returns a RobotObservation containing room, time, robot state, visit index, visible object identifiers, obstacle presence, temperature, humidity, transition accessibility, battery and a validity flag. The detector converts out-of-bound or changed observations into typed anomalies. Valid detections can update the Digital Twin and produce alerts with recommended actions. At the end, a report joins route, observations, detections, Twin revisions and execution status.

This ordering prevents later stages from consulting hidden simulator state. The detector receives observations only; Digital Twin updates receive observations and detection decisions; robot feedback summarises these downstream artefacts. Ground truth is used after the run by the evaluator

to calculate TP, FP and FN. The distinction is enforced both by field allow-lists and tests that change provenance without changing detection, and that change sensor observations and require detection to change.

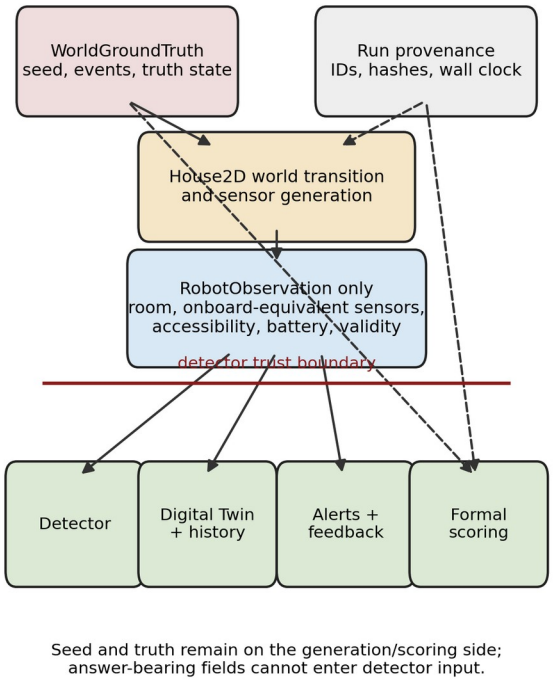

Figure 2. House-Sitter evidence flow. WorldGroundTruth and seed provenance generate the simulated world and remain on the scoring side; only RobotObservation crosses into detection, Digital Twin update, alerts and feedback.

3.9 House2D Experimental Backend

House2D is a seeded, pure-Python graph environment. Five named rooms are connected by a fixed door topology. Movement consumes simulated time and battery per traversed door; inspection has an additional battery cost. Events can affect temperature, humidity, visible objects, observation availability or transition accessibility. Sensor noise and dropout are generated deterministically from the seed. The backend records visit and route traces, final state and a separate ground-truth event ledger.

The design favours experimental control over physical detail. Rooms are graph nodes rather than geometry, route cost is an abstract distance/time unit, and sensing uses bounded numeric noise rather than a calibrated sensor model. A random obstacle position was initially specified even though this graph backend did not consume spatial positions. Before any RQ3 result was admitted, the variable was removed from the formal randomised dimensions and a correction record and hash were created. Remaining random dimensions pass consumption and sensitivity tests.

House2D version 1.1 is the experimental backend. Adding Create3ROS2Backend did not change its default profile, planner behaviour, seed mapping or formal results. It remains the only backend used for formal outcome claims. Calling it a research backend rather than a simulator of Create 3 avoids attributing physical fidelity it does not possess.

3.10 Observation and Ground-Truth Boundary

WorldGroundTruth contains the seed, event records, room state and door state needed to generate and score a scenario. Provenance contains run identifiers, hashes, paths and timestamps. Neither is a RobotObservation. The final RobotObservation allow-list excludes scenario seed, event identifiers, event types, truth labels, the event collection, simulation flags and physical-validation metadata. An observation with an extra field fails validation rather than passing an ignored clue to the detector.

The boundary corrected a serious pre-execution validity defect. Earlier development observations exposed active_event_identifiers and scenario_seed, which could make detection circular. The field split was implemented and independently reviewed before the final RQ3 run; all earlier uncommitted RQ3 attempts were discarded. RQ1 and RQ2 were preserved because their formal records did not use the House2D detector path. The formal provenance states that the correction occurred before any RQ3 record entered analysis and did not tune detector thresholds or outcomes.

Observation dropout is represented as an invalid or missing observation, whereas the pre-specified event ontology expected a missing_observation anomaly. The primary scoring preserves that distinction and consequently counts mismatches. The post-analysis audit additionally evaluates a post-hoc semantic remapping, but it is never substituted for the primary metric.

3.11 Anomaly Detection

The detector is deterministic. It checks observation validity, sensor thresholds, changes in visible objects, obstacle presence and transition accessibility. Its output includes the anomaly type, room, observation identifier, measured evidence and recommended response. It does not know which event the scenario generator intended. This makes false positives and false negatives meaningful rather than artefacts of comparing an event identifier with itself.

Threshold detection is deliberately simple. A single observation can cross a temperature or humidity limit because of event magnitude or bounded noise; there is no temporal filter or learned uncertainty model in the evaluated core. Dropout can suppress evidence that would otherwise support another anomaly. These simplifications are central to interpreting the primary F1 of 0.403: the experiment evaluates the implemented onboard-equivalent pipeline, not an ideal event decoder.

3.12 Digital Twin and History

The Digital Twin stores the latest trusted state for each room, revision identifiers and update provenance. A valid observation may initialise a room, confirm no change or produce a change record. Invalid observations are retained in evidence but do not overwrite trusted state. TwinHistory compares revisions and reports initialised rooms, unchanged rooms, confirmed changes and ignored observations.

Alerts refer to observation and Twin revision identifiers rather than truth event identifiers. Robot feedback can therefore explain that a measured humidity value exceeded a threshold or that a transition was inaccessible, without revealing how the simulator labelled the event. Twin correctness in RQ3 measures agreement between the resulting trusted state and ground truth at the defined evaluation point; it should not be interpreted as a general measure of Digital Twin fidelity.

3.13 Routing

The routing component operates on the House2D door graph. Given a start room and requested room set, it computes legal shortest paths and selects a lower-cost visit order. Explicit user order is preserved when semantically required; unordered room requests can be optimised. Every consecutive transition is checked against the topology. A blocked edge discovered during execution causes bounded termination rather than route teleportation or silent success.

RQ3 compares the optimised route with a route-optimisation-disabled condition using the same held-out seed and scenario ground truth. Both conditions execute in House2D because route ordering can safely be changed while retaining the same events and observations. The paired design estimates the within-scenario route-cost effect. It does not assume that a shorter route must increase mission completion; that is tested separately.

3.14 Robot Feedback and Demonstration Interface

Robot feedback translates artefacts into concise status rather than exposing implementation logs. It reports whether planning, verification, resource admission and execution proceeded; identifies the first failure; lists observations and detected anomalies; and links alerts to Digital Twin revisions. A local browser interface visualises the same pipeline, including capability exploration, TaskSpec, verification explanation, confirmation and replay.

The interface is a research demonstration, not a remote robot controller. During automated validation it runs locally with simulated data. It does not bypass verification, and no physical motion command is published. Archived Gazebo and earlier three-dimensional demo material are not evidence for the formal evaluation claims and are not used in the results figures.

3.15 ROS 2 / Create 3 Deployment Backend

RobotBackend defines the common high-level operations required by RaPToR-Lite: obtain an observation, execute a supported action, expose capability information, stop and report failures. House2DBackend implements these operations with deterministic graph state. Create3ROS2Backend maps them to live ROS 2 entities. The abstraction is intentionally an application contract rather than a claim that every backend has identical sensing semantics.

At connection time, Create3ROS2Backend inspects the ROS graph for topic, action and service names and their types. Its observation adapter maps battery state, odometry, inertial measurement, hazards and dock state when compatible messages are present. Motion capabilities are exposed for documented operations such as drive distance, rotate angle, drive arc and navigate to position when the corresponding action type is discovered. Dock and undock are similarly conditional. Emergency stop is mapped explicitly. A velocity topic may be discovered for diagnostics, but RaPToR-Lite does not expose unrestricted velocity publication as a high-level planner skill.

This design follows the typed topic, service and action distinctions in ROS 2 [39,48]. It also treats QoS mismatch, missing messages and stale timestamps as communication failures rather than “no anomaly” [47]. Failures cover ROS unavailable, robot unavailable, topic/action/type absence, timeout, stale sensor data, action cancellation, communication error, low battery, hazard and missing stop or return/dock capability. No such failure falls back to a successful House2D result.

Create 3 does not intrinsically know that a pose is “kitchen”. Named-room navigation is unavailable without a NavigationProvider. An optional Nav2 provider can expose it only when localisation, NavigateToPose and a valid room-to-waypoint mapping exist [46]. This keeps Gazebo and Nav2 outside the experimental core while leaving a legitimate deployment extension point.

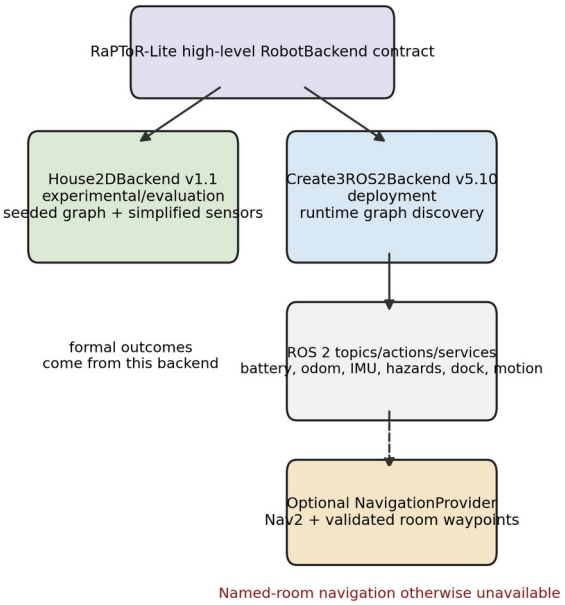

Figure 3. Backend boundary. House2D supplies deterministic experimental semantics; Create3ROS2Backend exposes only graph-discovered ROS 2 interfaces. Named rooms require an optional validated NavigationProvider and are otherwise unavailable.

The backend was tested with fake graph introspection, messages, services and action clients. A non-hardware ROS 2 Jazzy smoke test created an isolated local discovery node with no robot graph; the readiness report correctly listed every robot capability as unavailable and did not send an action or command. The implementation targets the official Create 3 ROS 2 API and networking model [32,33], but physical Create 3 operation, sensor accuracy, dock behaviour, Nav2 localisation and real-house safety were not tested. The accurate status is therefore: implemented deployment backend; interface/mock tested; physical robot validation not performed.

4. EVALUATION

4.1 Research Questions and Evaluation Logic

The three research questions separate decision mediation, language translation and integrated behaviour. RQ1 treats a materialised task case as the unit of analysis and asks whether capability grounding and verification lead to the correct accept, reject or clarify decision. RQ2 treats an utterance as the micro unit but groups uncertainty and comparisons by semantic target because eight forms express each target. RQ3 treats a held-out seed as the confirmatory unit and pairs system conditions on the same generated scenario.

This separation avoids using an end-to-end outcome to explain every component. A task may receive the correct accept decision yet differ from the oracle in a metadata field; it may produce a correct TaskSpec yet fail because a route is blocked; a mission may stop safely while anomaly detection remains inaccurate. Primary and secondary metrics preserve these distinctions rather than collapsing them into one “reliability” score.

4.2 Experimental Integrity

The research core, House2D configuration, schemas and backend identities were fixed before formal evaluation. Pilot outputs were stored under separate pilot directories, marked pilot_only=true, and excluded from formal statistics. Formal products are marked phase6_formal=true and contain the protocol and analysis-plan identifiers, Git revision, condition, seed or paired case identifier and raw outcome. Raw records were not altered during analysis; derived tables and plots are separate artefacts.

Protocol revision was permitted only before the affected formal data existed. Revision 1 materialised previously underspecified RQ1 cases, RQ2 utterances and RQ3 seeds. Revision 2 stated that unsafe RQ1 and resource-policy ablations were non-executing counterfactual decisions. A further pre-execution validity revision replaced system-derived labels with an independent oracle. Replay normalisation then excluded only volatile provenance, such as wall-clock timestamps, paths and run identifiers, from scientific equality. Finally, the House2D observation correction removed ground-truth fields and unused obstacle-position randomisation before any RQ3 record was admitted. None of these changes altered hypotheses, sample sizes, labels after observation, primary metrics or algorithm thresholds in response to results.

The protocol, analysis plan, pre-RQ3 correction and replay-normalisation addendum are versioned and recorded in Appendix A. These identifiers allow the result records to be matched to the exact pre-analysis materials without placing engineering hashes in the main evaluation narrative.

4.3 Independent Ground-Truth Oracles

RQ1 contains 240 static labelled cases: 48 each valid, invalid, unsafe, unsupported and ambiguous. Every case stores an actual language request or TaskSpec, expected decision, issue category, ground-truth reason and, where applicable, expected repair. Labels follow an explicit independent rule set. For example, “Inspect the garage” is unsupported by the declared House2D profile, a motion TaskSpec without a return is unsafe, and “Inspect a room” requires clarification.

RQ2 contains 40 semantic targets crossed with eight actual language forms. Each record stores the complete utterance and an independently materialised expected decision, intent and normalised TaskSpec, or a static clarify/reject label. The oracle module does not import OfflineHouseSitterPlanner, Verifier or the detector. An abstract-syntax-tree dependency guard and exact corpus checks enforce this property.

RQ3's oracle maps each pre-specified seed to direct House2D scenario events without calling the detector. It records room, event type and parameter, sensor-noise bound,

dropout and initial battery. Development seeds 61000–61099 and held-out seeds 62000–62299 are disjoint. The generator is deterministic, and all paired conditions receive the same ground truth. Consumption and sensitivity tests require every formal random dimension to affect a backend input or observable outcome. Ground truth scores the resulting observations and detections but never enters RobotObservation.

4.4 RQ1 Protocol

The 240 cases are evaluated under three paired conditions, producing 720 decision records. The full system uses both Capability Registry and Verifier. The capability-grounding ablation suppresses grounding issues while retaining the same proposed task and other checks. The verifier ablation suppresses verifier issues. Each record stores mutually exclusive would_accept, would_reject and would_clarify, counterfactual issues and correctness against the static label. executor_invoked is false for every condition.

Primary outcomes include decision correctness, acceptance and rejection precision/recall/F1, unsafe-catch rate and false-rejection rate where defined. The paired effect is the full-minus-ablation absolute difference. Exact McNemar tests operate on discordant paired correctness, and Benjamini–Hochberg correction controls the pre-specified comparison family. Metrics not defined for a condition are reported as N/A rather than reconstructed from absent events.

4.5 RQ2 Corpus and Protocol

Each of the 40 targets appears as canonical, paraphrase, synonym, explicit-order, unordered-rooms, ambiguity, unsupported-wrapper and unsafe-or-verifier-bypass language. The transformation name describes the surface form, not necessarily the target label. For example, an "unsupported" wrapper can say "within declared capabilities only" around a valid patrol; the independent semantic target remains accept. This design tests whether distracting language changes the underlying interpretation.

Micro metrics use all 320 utterances. Because the eight forms of a target are not independent semantic samples, macro means and bootstrap confidence intervals resample the 40 target groups. Decision correctness measures accept/reject/clarify agreement; intent correctness measures the normalised activity; TaskSpec exact match compares the complete normalised structure; end-to-end correctness requires the pre-specified combination of correct decision, intent, TaskSpec and verifier outcome. Language-form slices are secondary descriptive results.

4.6 RQ3 Stochastic Protocol

RQ3 uses 400 fixed seeds under three paired conditions: full system, route optimisation disabled, and resource-policy counterfactual. The 100 development seeds generate 300 logical development records used only for implementation checks. The 300 held-out seeds generate 900 confirmatory logical records. Forty held-out full-system cases are selected by the pre-specified replay rule, giving 1,240 logical records/checks overall. Development outcomes never enter the reported success rate, F1, confidence intervals, effect sizes or p-values.

The full condition executes the verified task in House2D. The route-disabled condition also executes, using the same scenario but an unoptimised legal order. The resource condition is read-only: it records whether execution would be attempted without policy and whether this would violate battery plus safe-return requirements. A full-policy defer is not executed in the counterfactual condition.

The randomised dimensions are room, direct event type and event parameter, sensor-noise bound, observation dropout and initial battery. Event timing and duration are excluded because the experimental backend has no honest input for them. Obstacle position is fixed provenance and excluded as a tested factor because House2D has no spatial obstacle model. The direct events comprise the monitoring phenomena the detector can encounter, including high temperature, high humidity, changed/missing objects, unexpected obstacle, missing observation and blocked transition.

Mission outcome is partitioned into four mutually exclusive classes: completed; safe defer before execution; safe blocked-route termination after execution begins; and genuine unexpected/system failure. The "execution-level safe outcome" rate counts the first three as bounded outcomes, but is never described as task success. Monitoring uses event TP, FP and FN plus precision, recall and mean per-seed F1. Twin correctness, ground-truth leakage, route cost, resource prevention and replay equality are separate outcomes.

4.7 Statistical Analysis

Binary rates are reported as exact counts and 95% Wilson-score confidence intervals [45,58]. RQ2 target-grouped macro intervals and paired route effects use deterministic non-parametric bootstrap resampling [17]. Paired binary comparisons use exact McNemar tests [43]. The pre-specified primary family is corrected using Benjamini–Hochberg false-discovery-rate control at q=0.05 [5]. Every principal comparison includes an absolute effect: percentage-point decision difference, paired route-cost difference, or percentage-point counterfactual prevention.

No significance threshold is used to hide negative results. Distributional summaries include n, mean, standard deviation, median, interquartile range and selected percentiles where present in the formal analysis. Failure taxonomy is defined before interpretation: unsupported false accept, missing timeout, invalid failure policy, missing safe return, unknown schema field, decision/intent/TaskSpec language error, detector TP/FP/FN, Twin mismatch, feedback leakage, route termination and reproducibility mismatch.

4.8 RQ1 Results: Capability Grounding and Verification

Condition	Correctness, class F1 and paired comparison
Full	240/240; 100.0% (95% CI 98.4–100.0); accept/reject F1 1.000/1.000; reference
No grounding	192/240; 80.0% (95% CI 74.5–84.6); accept/reject F1 0.667/0.800; full effect +20.0 pp; p=7.11e-15; q=1.07e-14
No verifier	160/240; 66.7% (95% CI 60.5–72.3); accept/reject F1 0.545/0.615; full effect +33.3 pp; p=1.65e-24; q=4.96e-24

Table 1. RQ1 paired decision results on the same 240 pre-specified cases. Effect is full-system correctness minus the named ablation. Unsafe and unsupported tasks were evaluated as non-executing counterfactual decisions.

The full system made the correct accept, reject or clarify decision on all 240 cases: 100.0%, with a 95% Wilson interval of 98.4–100.0%. Acceptance F1 and rejection F1 were both 1.000, and all 48 unsafe cases were caught. This is complete performance on the pre-specified, balanced benchmark, not evidence of 100% reliability in unrestricted language or a physical environment.

Removing capability grounding reduced correctness to 192/240 (80.0%, 95% CI 74.5–84.6%), an absolute full-system advantage of 20.0 percentage points. The 48 discordant cases were all unsupported-capability requests admitted by the ablation. Acceptance F1 was 0.667 and rejection F1 0.800. Exact McNemar p was 7.11×10^{-15} and BH-adjusted q was 1.07×10^{-14}.

Removing verification reduced correctness to 160/240 (66.7%, 95% CI 60.5–72.3%), a 33.3-point full-system advantage. Its 80 errors comprised 16 missing-timeout cases, 16 invalid failure-policy cases, 32 missing-safe-return cases and 16 unknown-field cases. Acceptance F1 was 0.545 and rejection F1 0.615. McNemar p was 1.65×10^{-24} and adjusted q was 4.96×10^{-24}. Both primary paired comparisons are significant after correction.

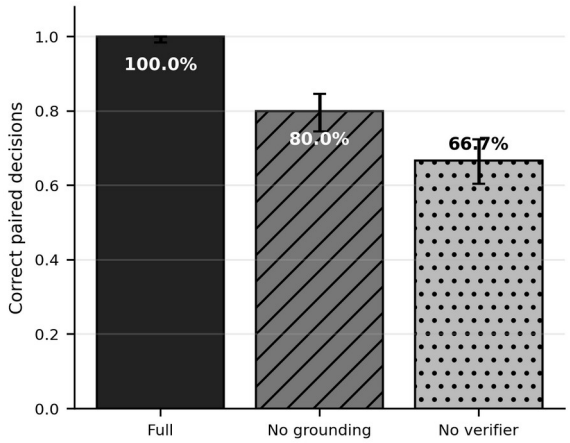

Figure 4. RQ1 decision correctness under paired conditions. Error bars are 95% Wilson intervals. Full-system improvements of 20.0 and 33.3 percentage points are measured on the same 240 cases; the 100% result is limited to this controlled benchmark.

The results answer RQ1 within the benchmark: capability grounding prevents unsupported task admission, while explicit verification prevents a larger set of structural and safety defects. The categories are complementary rather than interchangeable. A representative success is rq1-unsupported-001, where “Inspect the garage” is rejected because the room is absent from the profile. A representative verifier-only catch is rq1-unsafe-001, whose movement and inspection steps omit safe return. The full system rejects it before execution and can propose a separately verified repair, but the formal raw records do not provide a repair-success rate.

4.9 RQ2 Results: Natural-Language Reliability

Metric	Micro result and 40-target grouped interval
Decision	315/320; 98.4% (micro 95% CI 96.4–99.3); grouped 96.9–99.7%
Intent	286/320; 89.4% (micro 95% CI 85.5–92.3); grouped 83.1–94.1%
TaskSpec exact	272/320; 85.0% (micro 95% CI 80.7–88.5); grouped 73.8–94.4%
End-to-end	253/320; 79.1% (micro 95% CI 74.3–83.2); grouped 67.8–88.8%

Table 2. RQ2 reliability for 320 utterances and target-grouped macro analysis over 40 semantic targets. Micro intervals are Wilson intervals; grouped intervals bootstrap semantic targets, not individual paraphrases.

Decision correctness was 315/320 (98.4%, micro 95% CI 96.4–99.3%). Intent correctness was 286/320 (89.4%, 85.5–92.3%), TaskSpec exact match was 272/320 (85.0%, 80.7–88.5%), and end-to-end correctness was 253/320 (79.1%, 74.3–83.2%). Target-grouped macro means are identical because every target contributes eight forms, but the pre-specified bootstrap intervals are wider: 96.9–99.7% for decision, 83.1–94.1% for intent, 73.8–94.4% for exact match and 67.8–88.8% for end-to-end correctness.

Language-form end-to-end rates were 85.0% for canonical, 77.5% paraphrase, 55.0% synonym, 82.5% explicit order, 85.0% unordered rooms, 77.5% ambiguity, 85.0% unsupported wrappers and 85.0% unsafe/bypass wrappers. Synonyms are the clear weak slice. This result should not be read as a broad estimate of English synonym understanding: each slice contains one form for each of 40 controlled targets, and the templates themselves are fixed.

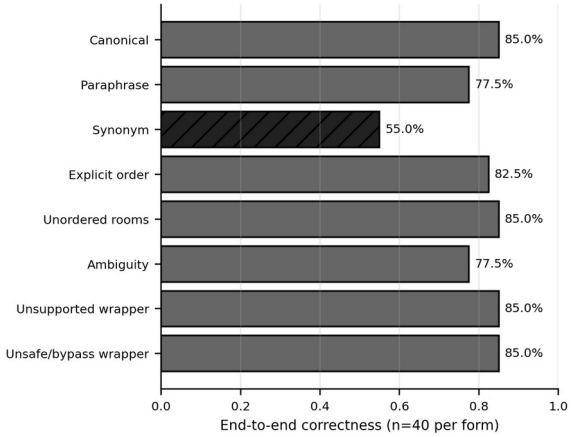

Figure 5. RQ2 end-to-end correctness by language form (40 utterances per form). The synonym slice is 55.0%; slices are descriptive and share semantic targets.

The errors reveal three patterns. First, lexical precedence over-generalises "monitoring task". For target-01-synonym, "Could you perform the following monitoring task: Inspect the living room", the expected intent is inspect but the planner selects patrol. Similar errors affect several inspect targets. Second, decisions can be correct while canonical structure differs. "Inspect the bathroom for humidity changes" is accepted with the correct intent, but the produced TaskSpec does not exactly preserve the oracle's checks metadata. Route-order and baseline metadata account for other exact mismatches. Third, wrappers can confuse boundary decisions. target-05-unsupported is a valid whole-house patrol prefixed with "Within declared capabilities only"; the planner clarifies rather than accepting. Conversely, target-36-synonym, "Use an unsupported sensor in the bedroom", is incorrectly accepted as patrol. These are not hidden by the high aggregate decision rate.

The distinction between decision and exact mapping is therefore substantive. Users would rarely see an unsafe execution in this benchmark because the Verifier remains downstream, but a semantically wrong accepted TaskSpec can still inspect the wrong scope or omit a requested check. RQ2 answers that constrained language is reliable for the coarse accept/reject/clarify decision, but less reliable for canonical intent and complete executable structure.

4.10 RQ3 Results: Stochastic House-Sitter Reliability

Metric	Held-out result, interval/effect and claim boundary
Mission completed	185/300; 61.7% (95% CI 56.1–67.0); task completion
Safe defer	66/300; 22.0% (17.7–27.0); not completion
Safe blocked route	49/300; 16.3%; mutually exclusive; not completion
Genuine failure	0/300 observed; no general reliability claim
Detection	P=0.486; R=0.515; primary F1=0.403 (bootstrap 0.350–0.460)
Twin correct	221/300; 73.7% (68.4–78.3); House2D state
Feedback leakage	0/300 (0.0–1.3%); observed rate
Route cost	full minus disabled=-4.737 (paired -5.186 to -4.268); completion effect 0 pp
Resource prevention	66/300; 22.0% (17.7–27.0); read-only counterfactual
Scientific replay	40/40 matched; same implementation and environment

Table 3. RQ3 confirmatory results from 300 held-out seeds. Development seeds are excluded. "Safe outcome" includes bounded defer and termination and is not mission success. Resource results are read-only counterfactuals.

Of 300 full-system held-out missions, 185 completed (61.7%, 95% CI 56.1–67.0%), 66 were safely deferred before execution (22.0%, 17.7–27.0%), and 49 began but terminated safely when a route transition was blocked (16.3%). There were no genuine unexpected or system failures. These categories are mutually exclusive and sum to 300. The resulting execution-level safe-outcome rate is 300/300, but mission completion remains 61.7%; describing this as 100% task success or general reliability would be incorrect.

There were 69 scenarios whose ground truth included a blocked-transition event. Twenty also had battery low enough that resource policy deferred them before the transition could be encountered. The remaining 49 reached the obstruction and terminated with the blocked-route outcome. None completed. Thus the event count of 69 and outcome count of 49 answer different questions: occurrence in ground truth versus terminal outcome after admission.

Monitoring was the weakest integrated component. Across held-out seeds the detector produced 121 true positives, 128 false positives and 114 false negatives. Aggregate event precision was 0.486 and recall 0.515. The primary mean per-seed F1 was 0.403 with 95% bootstrap CI 0.350–0.460. Digital Twin correctness was 221/300 (73.7%, 68.4–78.3%). Feedback ground-truth leakage was 0/300, with an upper Wilson bound of 1.26%. The absence of observed leakage supports the implemented boundary test; it does not prove that every future interface is leak-free.

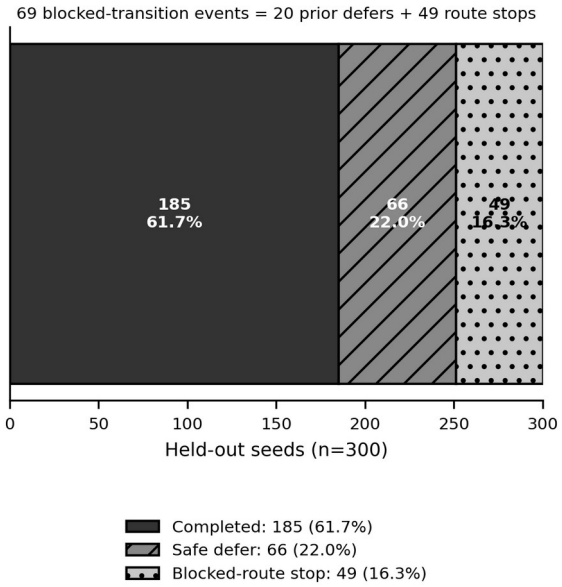

Figure 6. Mutually exclusive RQ3 held-out outcomes. Mission completion was 61.7%; safe defer and safe blocked-route termination are bounded safety outcomes, not completed missions. The 69 blocked-transition events comprise 20 low-battery defers and 49 route terminations.

Route optimisation reduced route cost by a paired mean of 4.737 route units (full minus disabled = -4.737, 95% bootstrap CI -5.186 to -4.268). The interval excludes zero, providing clear evidence of a route-cost benefit in this topology. Mission completion did not change: there were no discordant completion pairs, McNemar p=1.0 and BH q=1.0. The optimisation therefore improved efficiency without overcoming battery admission or blocked transitions.

Resource policy safely deferred 66/300 scenarios. In each, the read-only no-policy condition would have attempted a mission that violated the pre-specified battery and safe-return constraint. The prevention effect is therefore 22.0 percentage points (95% Wilson CI 17.7–27.0%). This is counterfactual unsafe-attempt prevention; no unsafe task was actually executed under the ablation.

Representative cases show the distinction among outcomes. Held-out seed 62000 contained high humidity and completed with a correct detection and Twin update. Seed 62001 began with 7% battery and was safely deferred. Seed 62012 encountered a blocked transition, retained its collected detection evidence and terminated before reaching the inaccessible bedroom. Such bounded failures are useful behaviour, but the latter two remain non-completions.

Scientific replay succeeded for all 40 selected held-out cases. Replay equality includes seed, condition, scenario input, TaskSpec, decisions, route, observations, detections, Twin state and outcome. It excludes wall-clock start/end time, run identifier and path metadata. Tests confirm that changing any route, observation, detection or outcome fails equality, while changing provenance timestamps does not. The result demonstrates deterministic replay of this software experiment, not replication on independent hardware.

4.11 Post-hoc Exploratory Sensitivity

The primary event ontology distinguishes the injected observation_dropout event from the detector's missing_observation label. A post-hoc sensitivity analysis treats these labels as semantically equivalent while leaving all other matching unchanged. Mean per-seed F1 rises from the primary 0.403 to 0.629 (95% bootstrap CI approximately 0.577–0.681), a difference of about +0.226; 110/300 seeds are affected.

This analysis is informative because it locates a substantial part of the error in ontology alignment rather than sensor thresholding. It does not replace the primary result: the remapping was defined after inspecting the formal outcome and was not part of the pre-specified metric. The primary F1 of 0.403 remains the confirmatory answer. The sensitivity instead motivates future work on a shared anomaly ontology and missing-data semantics.

5. DISCUSSION

5.1 Capability Grounding and Verification

RQ1 provides the clearest evidence for the architectural claim. The same 240 cases were evaluated under three decision rules, so the 20.0- and 33.3-point effects are not explained by different task samples. Capability grounding removed one coherent failure class: unsupported requests that remained plausible at the schema level. Verification removed a broader set of mechanical and safety defects. The absence of ablation-only correct pairs, together with very small adjusted p-values, indicates that both layers added value on the controlled benchmark rather than merely trading one kind of decision error for another.

The result also shows why “grounding” should not be used as a synonym for “safety”. SayCan-style affordance grounding constrains a plan by what the robot can do [31]. RaPToR-Lite's profile plays a simpler but related role. Yet a supported motion skill can still be unsafe in a plan without a timeout or return. Conversely, verification cannot determine support accurately if it is given a fictional capability profile. A mediation layer needs both a truthful affordance description and explicit plan invariants.

The controlled nature of RQ1 is a material limitation. Cases are balanced across five known strata, issue categories correspond to implemented rules, and the language subset is constrained. Perfect full-system correctness therefore demonstrates internal consistency and coverage of the benchmark, not open-world robustness. New skills, combined defects, adversarial phrasing or a stale runtime profile could produce errors outside the tested distribution. The strongest defensible claim is that explicit grounding and verification substantially improved paired task-level decisions under the pre-specified cases.

The counterfactual design strengthens safety but narrows interpretation. Unsupported or unsafe ablation outputs were

never executed. Consequently, the experiment measures decisions that would have admitted bad tasks, not physical harm or downstream failure frequency. That boundary is preferable to running dangerous plans for realism, but should remain clear in any comparison with runtime-assurance work [2,15].

5.2 Natural-Language Decision Reliability versus Canonicalisation

RQ2 shows that coarse decision reliability and semantic fidelity are different. A 98.4% correct accept/reject/clarify rate may appear close to solved, but only 79.1% of utterances satisfy the full end-to-end criterion. Intent errors and 48 TaskSpec mismatches account for much of the gap. In practice, a user could receive an executable, verifier-approved task that is structurally safe yet does not encode the requested check or exact route semantics.

Synonym performance is particularly revealing. The form was designed to preserve semantic targets while changing wording, but end-to-end correctness fell to 55.0%. The phrase “monitoring task” often activated the broad patrol rule before the embedded inspection instruction. This is a canonicalisation failure: the parser finds familiar tokens but assigns the wrong normal form. The result accords with grounded-language literature that treats language as dependent on task and environment context rather than keyword substitution [6,56].

Exact match is intentionally strict. Some mismatches concern metadata such as requested checks or route ordering rather than a wholly different skill sequence. Strictness is useful because those fields affect what a household report means, but it can make semantically near-equivalent tasks failures. The independent oracle and static labels avoid circular scoring, while the separate decision and intent metrics show where disagreement occurs. A future evaluation could add a pre-registered semantic-equivalence metric, but introducing it after observing these results would obscure the pre-specified exact-match limitation.

The system's clarification behaviour is uneven. It correctly avoids many ambiguous tasks, yet can accept “Inspect a room” when surrounded by a synonym wrapper, and it sometimes clarifies a valid request carrying distracting capability language. Interactive dialogue could resolve these cases, as demonstrated by grounded human–robot dialogue research [57], but no user study or multi-turn clarification policy was evaluated here. The present result supports confirmation and visible TaskSpec review: a constrained parser should not be treated as an invisible compiler.

5.3 Safety versus Mission Completion

RQ3 makes a distinction that is often lost in robot reporting. Only 185 of 300 held-out missions completed. The other 115 were bounded by resource or route safety, producing an execution-level safe-outcome rate of 100%. The latter is positive evidence that the implemented failure paths behaved as specified in these scenarios, but it is not 100% mission success, reliability or usefulness.

Safe defer and blocked-route termination have different implications. A defer occurs before execution and protects the return reserve, while blocked-route termination occurs after some work may have been completed. The 69/49 reconciliation shows their interaction: twenty blocked-transition worlds were never entered because low battery took precedence. An outcome taxonomy that simply labels both as “failure” would hide the system's safety contribution; one that calls both “success” would hide the 38.3% non-completion rate. Reporting mutually exclusive outcomes preserves both facts.

There were no genuine unexpected/system failures in the held-out set. This indicates that timeouts, graph errors and model exceptions did not escape the bounded paths exercised by the formal scenarios. It does not estimate rare software failures outside 300 seeds, nor does it validate emergency behaviour on hardware. The Wilson interval for observed zero events still permits a non-zero population rate, and the simulator omits many physical causes of failure.

5.4 Monitoring Limitations

The primary detection F1 of 0.403 is the most important negative result. Precision and recall are both near 0.5, with substantial false-positive and false-negative counts. A House-Sitter that completes a route but reports unreliable anomalies has limited practical value. Digital Twin correctness of 73.7% inherits part of this weakness because the Twin is updated from the observation/detection pipeline.

Several mechanisms contribute. House2D uses single-visit thresholding with bounded noise, not calibrated sensor likelihoods or temporal filters. Dropout can remove evidence. Changed-object and missing-observation events require categorical matching. Most notably, the scenario generator and detector used different labels for observation dropout. The post-hoc equivalence analysis raises F1 to 0.629, showing that ontology alignment matters, but performance remains imperfect and the analysis is exploratory.

The correct response is not to replace the primary value. Instead, future work should define a shared pre-registered event ontology, distinguish “sensor produced no message” from “observation invalid”, and evaluate temporal confirmation. Real hardware would add sensor drift, QoS loss, occlusion and room-localisation uncertainty absent from House2D. Context-aware and smart-home systems likewise depend on explicit acquisition and uncertainty models [12,49]. The current detector is best understood as a transparent baseline.

The ground-truth separation repair improves validity without improving the detector. Removing event identifiers and seeds ensures that detections arise from onboard-equivalent fields. This correction occurred before RQ3 data entered analysis and was followed by sensitivity tests across every random dimension. It is an example of a scientifically necessary pre-analysis correction, not outcome tuning.

5.5 Route and Resource Policies

Route optimisation produced a clear efficiency effect: approximately 4.74 fewer route units, with a paired interval excluding zero. Pairing matters because house event, noise, dropout and battery are identical within each seed. The result establishes that the optimiser selects cheaper legal paths in the fixed topology. It did not improve completion, because the causes of non-completion were policy defer and blocked transitions that route ordering could not remove.

The resource policy prevented hypothetical unsafe attempts in 22.0% of held-out scenarios. Its practical value is admission control rather than increased task throughput. The result is particularly easy to overstate: the no-policy condition is read-only and never performs an unsafe run. It supports the claim that the rule would block an attempt violating the modelled battery reserve, not that 66 real or simulated robots were saved from battery exhaustion.

Both policies depend on simplified costs. House2D battery consumption and route units are deterministic abstractions. On a physical Create 3, surface, wheel slip, docking availability, battery health and navigation recovery would change cost. A deployment should estimate uncertainty, preserve a configurable margin and calibrate against logged hardware data rather than copy the House2D constants.

5.6 ROS 2 Deployment Implications

Create3ROS2Backend demonstrates that the task-level contract can be connected to a real middleware boundary without reusing Gazebo as the research mainline. Runtime discovery is important because ROS 2 systems are assembled graphs, not static product APIs. A disconnected action, incompatible type or QoS mismatch must make the related capability unavailable. This fail-closed behaviour is more defensible than exposing a skill from an expected name alone [39,47].

The mapping also shows a semantic gap. Create 3 provides base motion, hazard, odometry, inertial, dock and battery interfaces documented by iRobot [33]. It does not natively provide temperature, humidity or named household rooms. An external sensor source would be needed for environmental claims, and a NavigationProvider would need localisation plus legitimate waypoints for room navigation. Nav2 availability alone does not prove that “kitchen” resolves to a safe pose.

Interface/mock tests are necessary but insufficient deployment evidence. They verify type mapping, discovery, timeout, cancellation, stale data and explicit unavailability. The empty-graph smoke test verifies that the backend fails closed in a local ROS 2 environment. Neither exercises wireless loss, physical braking, dock alignment, hazard-sensor accuracy, localisation or household obstacles. Physical validation should begin with stationary discovery and observation, then controlled action tests with an operator and emergency-stop procedure; it is future work, not an implicit achievement of this dissertation.

5.7 Limitations

The principal limitation is simulation-only evaluation. House2D has a fixed five-room topology, abstract motion costs and simplified sensing. It cannot establish real navigation, sensor accuracy, physical safety or long-term operation. The Create 3 backend narrows the implementation gap but remains physical_robot_validated=false.

The language corpus is controlled: 320 utterances arise from 40 semantic targets and eight fixed forms. Grouped analysis avoids pretending that paraphrases are independent, but the sample still excludes open dialogue, speech recognition, accents, typos, anaphora and household-specific vocabulary. The synonym slice and exact-match failures show that even this restricted language is not solved. No participants evaluated clarity, trust, workload or confirmation usability, although HRI literature indicates these factors influence acceptance [25,28,40].

The RQ1 benchmark is balanced and rule-aligned. Its 100% full score should not be expected under combined defects or unknown skills. The ablations are counterfactual decisions rather than executing unsafe work. Formal safe-repair success is unavailable from the raw records. These restrictions limit claims but make the evaluation safer and more auditable.

RQ3 has only 300 confirmatory seeds in one topology and one detector configuration. The primary F1 is low, the dropout remapping is post-hoc, and Twin correctness is tied to the same observation model. Route efficiency is expressed in House2D units. The 40/40 replay result addresses deterministic scientific payload equality on the same implementation and environment, not independent replication or hardware repeatability.

Digital Twin terminology can imply more fidelity than implemented. The project stores revisioned room belief and provenance but does not model physics, forecast degradation or synchronise with a physical home. Security, privacy and retention were not evaluated despite their importance in smart-home deployments [22,59].

5.8 Future Work

The first priority is language canonicalisation. Parser rules should give the embedded imperative priority over generic wrapper phrases, use a shared synonym lexicon and expose ambiguous slot resolution through multi-turn clarification. Improvements should be evaluated on a newly held-out corpus with pre-specified equivalence criteria rather than retrofitted to the formal evaluation.

The second priority is monitoring. A pre-registered anomaly ontology should align generator, observation adapter and detector. Temporal filtering, calibrated uncertainty and explicit sensor-health state could reduce single-sample errors. A larger set of topologies and event combinations would test whether Twin correctness and route results generalise.

The third priority is staged physical validation. With a Create 3 available, testing should begin without motion: graph discovery, message type compatibility, QoS, timestamp freshness and battery/dock observations. Action tests should then use a bounded area and explicit operator authorisation. Named rooms should remain unavailable until localisation and waypoint validation succeed. Environmental monitoring would require sensors whose accuracy and placement are documented.

Finally, a human study should examine whether users understand accept, reject, clarify, defer and safe termination; whether TaskSpec confirmation prevents semantic errors; and whether explanations improve calibrated trust. Such evidence would connect the system-level contribution to the domestic HRI concerns that motivated the work.

6. CONCLUSION

This dissertation introduced RaPToR-Lite, a capability-grounded layer for turning constrained household language into typed, verified robot tasks. The architecture makes the execution boundary explicit: a planner proposes TaskSpec, a runtime profile declares available skills, a deterministic Verifier checks structure and safety, resource admission protects return reserve, and only a confirmed plan reaches a RobotBackend. The House-Sitter case integrates patrol, monitoring, route selection, Digital Twin history and feedback. House2D supports controlled evaluation; Create3ROS2Backend provides a graph-discovered deployment path without claiming hardware validation.

RQ1 showed that the mediation layers matter on the controlled benchmark. Full decisions were correct for 240/240 cases, improving by 20.0 points over removal of grounding and 33.3 points over removal of verification. RQ2 showed the boundary of that success: decision accuracy was 98.4%, but exact structured mapping was 85.0%, end-to-end correctness 79.1%, and synonym performance 55.0%. RQ3 showed bounded integrated behaviour rather than universal reliability. Mission completion was 61.7%; resource and route safeguards accounted for the remaining safe outcomes; route optimisation reduced cost but not non-completion; primary anomaly-detection F1 was 0.403; and all 40 scientific replays matched.

The central conclusion is therefore qualified. Explicit capability grounding and verification can materially improve task-level safety decisions in a controlled natural-language robot interface. They do not compensate for weak semantic canonicalisation, limited sensing or an inaccurate anomaly ontology. A reproducible experimental backend and a fail-closed ROS 2 adapter make these limits visible and create a credible path to future deployment, but physical Create 3 validation, real-house navigation and real-sensor safety remain unperformed.

ACKNOWLEDGMENTS

The author is grateful to their supervisor for guidance and support throughout this project.

REFERENCES

[1] M. Alaa, A. A. Zaidan, B. B. Zaidan, Mohammed Talal and M. L. M. Kiah. 2017. A Review of Smart Home Applications Based on Internet of Things. Journal of Network and Computer Applications 97, 48–65. https://doi.org/10.1016/j.jnca.2017.08.017

[2] Mohammed Alshiekh, Roderick Bloem, Rüdiger Ehlers, Bettina Könighofer, Scott Niekum and Ufuk Topcu. 2018. Safe Reinforcement Learning via Shielding. Proceedings of the AAAI Conference on Artificial Intelligence 32(1). https://doi.org/10.1609/aaai.v32i1.11797

[3] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Niko Sünderhauf, Ian Reid, Stephen Gould and Anton van den Hengel. 2018. Vision-and-Language Navigation: Interpreting Visually-Grounded Navigation Instructions in Real Environments. 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 3674–3683. https://doi.org/10.1109/CVPR.2018.00387

[4] Brenna D. Argall, Sonia Chernova, Manuela Veloso and Brett Browning. 2009. A Survey of Robot Learning from Demonstration. Robotics and Autonomous Systems 57(5), 469–483. https://doi.org/10.1016/j.robot.2008.10.024

[5] Yoav Benjamini and Yosef Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. Journal of the Royal Statistical Society: Series B 57(1), 289–300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x

[6] Yonatan Bisk, Ari Holtzman, Jesse Thomason, Jacob Andreas, Yoshua Bengio, Joyce Chai, Mirella Lapata, Angeliki Lazaridou, Jonathan May, Aleksandr Nisnevich, Nicolas Pinto and Joseph Turian. 2020. Experience Grounds Language. Proceedings of EMNLP 2020, 8718–8735. https://doi.org/10.18653/v1/2020.emnlp-main.703

[7] Fabio Bonsignorio, Angel P. del Pobil and Enrica Zereik. 2025. Towards Reproducible Robotics Research. Nature Machine Intelligence 7. https://doi.org/10.1038/s42256-025-01114-7

[8] Anthony Brohan et al. 2023. RT-1: Robotics Transformer for Real-World Control at Scale. Robotics: Science and Systems XIX. https://doi.org/10.15607/RSS.2023.XIX.025

[9] Rodney A. Brooks. 1986. A Robust Layered Control System for a Mobile Robot. IEEE Journal on Robotics and Automation 2(1), 14–23. https://doi.org/10.1109/JRA.1986.1087032

[10] Marie Chan, Eric Campo, Daniel Estève and Jean-Yves Fourniols. 2009. Smart Homes–Current Features and Future Perspectives. Maturitas 64(2), 90–97. https://doi.org/10.1016/j.maturitas.2009.07.014

[11] Michele Colledanchise and Petter Ögren. 2018. Behavior Trees in Robotics and AI: An Introduction. CRC Press, Boca Raton, FL. https://doi.org/10.1201/9780429489105

[12] Diane J. Cook, Aaron S. Crandall, Brian L. Thomas and Narayanan C. Krishnan. 2013. CASAS: A Smart Home in a Box. Computer 46(7), 62–69. https://doi.org/10.1109/MC.2012.328

[13] Kerstin Dautenhahn, Sarah Woods, Christina Kaouri, Michael L. Walters, Kheng Lee Koay and Iain Werry. 2005. What Is a Robot Companion–Friend, Assistant or Butler? 2005 IEEE/RSJ International Conference on Intelligent Robots and Systems, 1192–1197. https://doi.org/10.1109/IROS.2005.1545189

[14] L. C. De Silva, Chamintha Morikawa and Iskandar M. Petra. 2012. State of the Art of Smart Homes. Engineering Applications of Artificial Intelligence 25(7), 1313–1321. https://doi.org/10.1016/j.engappai.2012.05.002

[15] Ankush Desai, Shromona Ghosh, Sanjit A. Seshia, Natarajan Shankar and Ashish Tiwari. 2019. SOTER: A Runtime Assurance Framework for Programming Safe Robotics Systems. 2019 IEEE/IFIP International Conference on Dependable Systems and Networks, 138–150. https://doi.org/10.1109/DSN.2019.00027

[16] Danny Driess et al. 2023. PaLM-E: An Embodied Multimodal Language Model. Proceedings of the 40th International Conference on Machine Learning, PMLR 202, 8469–8488. https://proceedings.mlr.press/v202/driess23a.html (accessed 10 August 2026).

[17] Bradley Efron and Robert J. Tibshirani. 1994. An Introduction to the Bootstrap. Chapman & Hall/CRC, New York. https://doi.org/10.1201/9780429246593

[18] Abdulmotaleb El Saddik. 2018. Digital Twins: The Convergence of Multimedia Technologies. IEEE MultiMedia 25(2), 87–92. https://doi.org/10.1109/MMUL.2018.023121167

[19] David Feil-Seifer and Maja J. Matarić. 2005. Socially Assistive Robotics. 9th International Conference on Rehabilitation Robotics, 465–468. https://doi.org/10.1109/ICORR.2005.1501143

[20] Terrence Fong, Illah Nourbakhsh and Kerstin Dautenhahn. 2003. A Survey of Socially Interactive Robots. Robotics and Autonomous Systems 42(3–4), 143–166. https://doi.org/10.1016/S0921-8890(02)00372-X

[21] Jodi Forlizzi and Carl DiSalvo. 2006. Service Robots in the Domestic Environment: A Study of the Roomba Vacuum in the Home. Proceedings of the 1st ACM SIGCHI/SIGART Conference on Human-Robot Interaction, 258–265. https://doi.org/10.1145/1121241.1121286

[22] Aidan Fuller, Zhong Fan, Charles Day and Chris Barlow. 2020. Digital Twin: Enabling Technologies, Challenges and Open Research. IEEE Access 8, 108952–108971. https://doi.org/10.1109/ACCESS.2020.2998358

[23] Malik Ghallab, Dana Nau and Paolo Traverso. 2004. Automated Planning: Theory and Practice. Morgan Kaufmann, San Francisco, CA. https://www.sciencedirect.com/book/9781558608566/automated-planning (accessed 10 August 2026).

[24] Edward Glaessgen and David Stargel. 2012. The Digital Twin Paradigm for Future NASA and U.S. Air Force Vehicles. 53rd AIAA Structures, Structural Dynamics and Materials Conference. https://doi.org/10.2514/6.2012-1818

[25] Michael A. Goodrich and Alan C. Schultz. 2007. Human-Robot Interaction: A Survey. Foundations and Trends in Human-Computer Interaction 1(3), 203–275. https://doi.org/10.1561/1100000005

[26] Michael Grieves and John Vickers. 2017. Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems. In Transdisciplinary Perspectives on Complex Systems, Springer, 85–113. https://doi.org/10.1007/978-3-319-38756-7_4

[27] Odd Erik Gundersen and Sigbjørn Kjensmo. 2018. State of the Art: Reproducibility in Artificial Intelligence. Proceedings of the AAAI Conference on Artificial Intelligence 32(1). https://doi.org/10.1609/aaai.v32i1.11503

[28] Marcel Heerink, Ben Kröse, Vanessa Evers and Bob Wielinga. 2010. Assessing Acceptance of Assistive Social Agent Technology by Older Adults: The Almere Model. International Journal of Social Robotics 2, 361–375. https://doi.org/10.1007/s12369-010-0068-5

[29] Thomas M. Howard, Stefanie Tellex and Nicholas Roy. 2014. A Natural Language Planner Interface for Mobile Manipulators. 2014 IEEE International Conference on Robotics and Automation, 6652–6659. https://doi.org/10.1109/ICRA.2014.6907841

[30] Wenlong Huang, Pieter Abbeel, Deepak Pathak and Igor Mordatch. 2022. Language Models as Zero-Shot Planners: Extracting Actionable Knowledge for Embodied Agents. Proceedings of the 39th International Conference on Machine Learning, PMLR 162, 9118–9147. https://proceedings.mlr.press/v162/huang22a.html (accessed 10 August 2026).

[31] Brian Ichter et al. 2023. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. Proceedings of the 6th Conference on Robot Learning, PMLR 205, 287–318. https://proceedings.mlr.press/v205/ichter23a.html (accessed 10 August 2026).

[32] iRobot Education. 2024. Create 3 Networking Configuration. Create 3 Documentation. https://iroboteducation.github.io/create3_docs/setup/network-config/ (accessed 10 August 2026).

[33] iRobot Education. 2024. Create 3 ROS 2 Actions. Create 3 Documentation. https://iroboteducation.github.io/create3_docs/api/ros2/ (accessed 10 August 2026).

[34] Thomas Kollar, Stefanie Tellex, Deb Roy and Nicholas Roy. 2010. Toward Understanding Natural Language Directions. 2010 ACM/IEEE International Conference on Human-Robot Interaction, 259–266. https://doi.org/10.1109/HRI.2010.5453186

[35] Hadas Kress-Gazit, Morteza Lahijanian and Vasumathi Raman. 2018. Synthesis for Robots: Guarantees and Feedback for Robot Behavior. Annual Review of Control, Robotics, and Autonomous Systems 1, 211–236. https://doi.org/10.1146/annurev-control-060117-104838

[36] Werner Kritzinger, Matthias Karner, Georg Traar, Jan Henjes and Wilfried Sihn. 2018. Digital Twin in Manufacturing: A Categorical Literature Review and Classification. IFAC-PapersOnLine 51(11), 1016–1022. https://doi.org/10.1016/j.ifacol.2018.08.474

[37] Jacky Liang, Wenlong Huang, Fei Xia, Peng Xu, Karol Hausman, Brian Ichter, Pete Florence and Andy Zeng. 2023. Code as Policies: Language Model Programs for Embodied Control. 2023 IEEE International Conference on Robotics and Automation, 9493–9500. https://doi.org/10.1109/ICRA48891.2023.10160591

[38] Matt Luckcuck, Marie Farrell, Louise A. Dennis, Clare Dixon and Michael Fisher. 2020. Formal Specification and Verification of Autonomous Robotic Systems: A Survey. ACM Computing Surveys 52(5), Article 100. https://doi.org/10.1145/3342355

[39] Steven Macenski, Tully Foote, Brian Gerkey, Chris Lalancette and William Woodall. 2022. Robot Operating System 2: Design, Architecture, and Uses in the Wild. Science Robotics 7(66), eabm6074. https://doi.org/10.1126/scirobotics.abm6074

[40] Matthew Marge et al. 2022. Spoken Language Interaction with Robots: Recommendations for Future Research. Computer Speech & Language 71, 101255. https://doi.org/10.1016/j.csl.2021.101255

[41] Yuya Maruyama, Shinpei Kato and Takuya Azumi. 2016. Exploring the Performance of ROS2. Proceedings of the 13th International Conference on Embedded Software, Article 5. https://doi.org/10.1145/2968478.2968502

[42] Cynthia Matuszek, Evan Herbst, Luke Zettlemoyer and Dieter Fox. 2013. Learning to Parse Natural Language Commands to a Robot Control System. Experimental Robotics, Springer Tracts in Advanced Robotics 88, 403–415. https://doi.org/10.1007/978-3-319-00065-7_28

[43] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. Psychometrika 12, 153–157. https://doi.org/10.1007/BF02295996

[44] Dipendra Kumar Misra, Jaeyong Sung, Kevin Lee and Ashutosh Saxena. 2014. Tell Me Dave: Context-Sensitive Grounding of Natural Language to Manipulation Instructions. Robotics: Science and Systems X. https://doi.org/10.15607/RSS.2014.X.005

[45] Robert G. Newcombe. 1998. Interval Estimation for the Difference Between Independent Proportions: Comparison of Eleven Methods. Statistics in Medicine 17(8), 873–890. https://doi.org/10.1002/(SICI)1097-0258(19980430)17:8<873::AID-SIM779>3.0.CO;2-I

[46] Open Navigation LLC. 2025. Navigation2 Concepts and NavigateToPose Action. Navigation2 Documentation. https://docs.nav2.org/ (accessed 10 August 2026).

[47] Open Robotics. 2025. ROS 2 Jazzy Documentation: Quality of Service Settings. ROS 2 Documentation. https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html (accessed 10 August 2026).

[48] Open Robotics. 2025. ROS 2 Jazzy Documentation: Topics, Services and Actions. ROS 2 Documentation. https://docs.ros.org/en/jazzy/Concepts/Basic.html (accessed 10 August 2026).

[49] Charith Perera, Arkady Zaslavsky, Peter Christen and Dimitrios Georgakopoulos. 2014. Context Aware Computing for The Internet of Things: A Survey. IEEE Communications Surveys & Tutorials 16(1), 414–454. https://doi.org/10.1109/SURV.2013.042313.00197

[50] Hans E. Plesser. 2018. Reproducibility vs. Replicability: A Brief History of a Confused Terminology. Frontiers in Neuroinformatics 11, 76. https://doi.org/10.3389/fninf.2017.00076

[51] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. Advances in Neural Information Processing Systems 36. https://proceedings.neurips.cc/paper_files/paper/2023/hash/d842425e4bf79ba039352da0f658a906-Abstract-Conference.html (accessed 10 August 2026).

[52] Sanjit A. Seshia, Dorsa Sadigh and S. Shankar Sastry. 2022. Toward Verified Artificial Intelligence. Communications of the ACM 65(7), 46–55. https://doi.org/10.1145/3503914

[53] Yoshiaki Shiokawa, Winnie Chen, Aditya Shekhar Nittala, Jason Alexander and Adwait Sharma. 2025. Beyond Vacuuming: How Can We Exploit Domestic Robots' Idle Time? Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems, Article 816. https://doi.org/10.1145/3706598.3714266

[54] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer and Dieter Fox. 2020. ALFRED: A Benchmark for Interpreting Grounded Instructions for Everyday Tasks. 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition, 10737–10746. https://doi.org/10.1109/CVPR42600.2020.01075

[55] Stefanie Tellex, Thomas Kollar, Steven Dickerson, Matthew R. Walter, Ashis Gopal Banerjee, Seth Teller and Nicholas Roy. 2011. Understanding Natural Language Commands for Robotic Navigation and Mobile Manipulation. Proceedings of the AAAI Conference on Artificial Intelligence 25(1), 1507–1514. https://doi.org/10.1609/aaai.v25i1.7979

[56] Stefanie Tellex, Nakul Gopalan, Hadas Kress-Gazit and Cynthia Matuszek. 2020. Robots That Use Language. Annual Review of Control, Robotics, and Autonomous Systems 3, 25–55. https://doi.org/10.1146/annurev-control-101119-071628

[57] Jesse Thomason, Aishwarya Padmakumar, Jivko Sinapov, Nick Walker, Yuqian Jiang, Harel Yedidsion, Justin Hart, Peter Stone and Raymond J. Mooney. 2019. Improving Grounded Natural Language Understanding through Human-Robot Dialog. 2019 International Conference on Robotics and Automation, 6934–6941. https://doi.org/10.1109/ICRA.2019.8794287

[58] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. Journal of the American Statistical Association 22(158), 209–212. https://doi.org/10.1080/01621459.1927.10502953

[59] Charlie Wilson, Tom Hargreaves and Richard Hauxwell-Baldwin. 2017. Benefits and Risks of Smart Home Technologies. Energy Policy 103, 72–83. https://doi.org/10.1016/j.enpol.2016.12.047

[60] Brianna Zitkovich et al. 2023. RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. Proceedings of the 7th Conference on Robot Learning, PMLR 229, 2165–2183. https://proceedings.mlr.press/v229/zitkovich23a.html (accessed 10 August 2026).

APPENDIX A. REPRODUCIBILITY AND ARTEFACT INDEX

The dissertation evidence pack records the Git baseline, protocol and analysis identifiers, RQ raw logical identifiers, results-manifest identifier, source-file identifiers and every table/figure dependency. Formal RQ1, RQ2 and RQ3 records remain outside the dissertation commit and are checked byte-for-byte before and after document generation. Pilot directories are explicitly excluded.

Protocol SHA-256: 25a16e15fc07c2c9d3c76e52de067ca47f09950ac532bbbf1f14e611753c2847. Analysis-plan SHA-256: fc1e1e1e20817435a80c0886715dcb25ce4ee3844e0ecf64f15c7189f34f9594. RQ3 implementation-correction SHA-256: cae4f103e4777485397ec3b239d108c428fc20c2ebea994d1aa0c0afc9f550f9. Replay-normalisation addendum SHA-256: 68dd987fe6d8a4f9fd1e06b281f119e75d2640a481bc977f06a9dcae0767d57a. Results-manifest SHA-256: dc249cfff4b8f9bab90173555fb9860f96b22381fee64ff792ad9599e5499dec.

RQ1 contains 720 records: 240 pair identifiers crossed with three decision conditions. RQ2 contains 320 records: 40 target identifiers crossed with eight language forms. RQ3 contains 1,200 records: 400 disjoint seeds crossed with three conditions, plus 40 scientific replay records. Confirmatory analysis filters RQ3 to the 300 held-out seeds and never combines the 100 development seeds with paper statistics.

Canonical replay retains all fields capable of altering a conclusion: seed, condition, scenario, TaskSpec, verification and resource decisions, route, observations, detections, Digital Twin state and outcome. It excludes volatile wall-clock timestamps, run identifiers and file-system paths while preserving them in provenance. Mutation tests demonstrate that changing route, observation, detection or outcome changes the scientific hash.

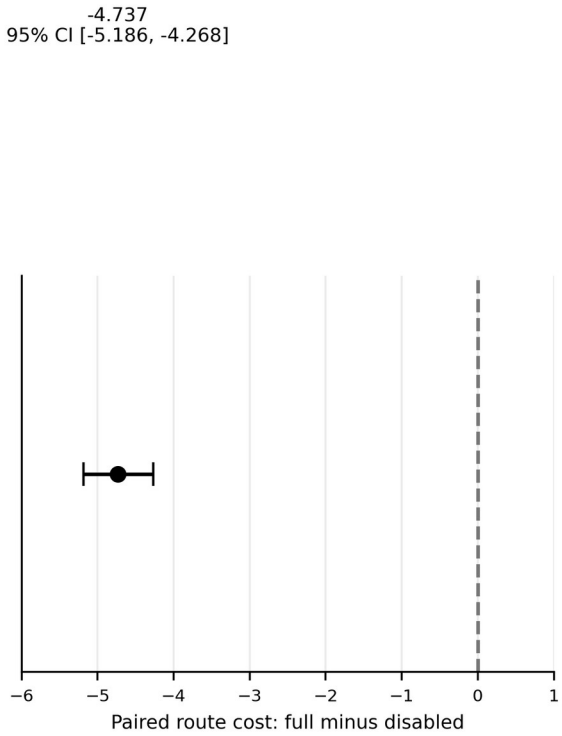

Figure A1. Supplementary paired route-cost comparison for the 300 held-out seeds. The pre-specified estimate is full minus route-disabled = -4.737 units (95% bootstrap CI -5.186 to -4.268); completion did not change.

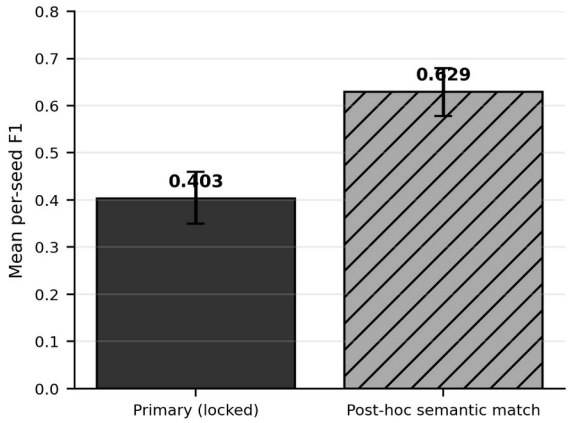

Figure A2. Supplementary comparison of the primary anomaly-detection F1 and the post-hoc dropout/missing-observation semantic sensitivity. The exploratory value does not replace the primary result.

APPENDIX B. CLAIM AND DEPLOYMENT BOUNDARIES

The results should be read with the following fixed distinctions. 100% in RQ1 means correct decisions on 240 balanced controlled cases. 100% execution-level safe outcome in RQ3 includes completed missions, safe defer and safe blocked-route termination; it is not task success. The confirmatory detection F1 is 0.403. The 0.629 value is post-hoc exploratory sensitivity. Resource ablation describes a counterfactual unsafe attempt and never executes the unsafe task. No formal percentage is available for safe-repair success.

House2D is an experimental backend with simplified graph motion and sensing. Create3ROS2Backend is an implemented deployment backend with runtime graph discovery, explicit unavailable capabilities and an optional NavigationProvider boundary. It has fake/mock integration tests and a non-hardware empty-graph ROS smoke test. No physical Create 3, physical safety, real-house localisation, Nav2 room navigation or real sensor accuracy was validated; physical_robot_validated=false remains part of the evidence and deployment report.